

Core Design Principles

A Merged Guide from GeeksforGeeks, Medium, and the Harvard Kempner Institute Software Engineering Handbook

Week 6 – Software Planning and Development | Prepared by: Vincent Onchieku Collins

A note on scope

SOLID, DRY, KISS, YAGNI, composition-over-inheritance, polymorphism-over-conditionals, basic dependency injection, the Law of Demeter, and the Boy Scout Rule are already covered in the Clean Coding Practices file and are not repeated here. This document focuses on the broader design principles — architectural, structural, and process-level — that sit alongside those.

1. What Are Software Design Principles, and Why Do They Matter?

Software Design Principles are a set of guidelines that help developers arrive at a good system design. In a typical project, writing code consumes only about 20–40% of total effort — the rest goes to reading and maintaining what was written. That makes design, not typing speed, the real driver of long-term cost.

It helps to separate three activities that are often blurred together. Software Architecture is the high-level decision-making — it determines how a system deals with performance, fault tolerance, scalability and reliability. Software Design is the lower-level counterpart — what each module does, class scope, method purposes. Coding is the implementation of that design in a programming language. Design is not coding, and coding is not design: design describes the logic that solves a problem, while code is simply the language used to express that logic.

There is no single, universally agreed definition of “good design,” but most experienced engineers converge on one idea: a good design is one where the cost and time of changing it — especially maintaining it — stays low. Since requirements and technology keep shifting, a design should be tested by subjecting it to change and seeing whether it holds up gracefully. Design principles are the guidelines experienced developers distilled from their own past mistakes, so that others don't have to relearn them the hard way.

Sources: Medium – “Software Design Principles Every Programmer Should Know”; GeeksforGeeks – “An Introduction to Software Development Design Principles.”

2. Ten Principles of Good Software Design

Beyond any single named principle, GeeksforGeeks identifies ten broad qualities that well-designed software exhibits across the whole design process:

Principle	What It Means
No “Tunnel Vision”	The design process shouldn't focus only on achieving the immediate aim while ignoring other effects and consequences.
Traceable to the Analysis Model	The design must be traceable back to requirements, satisfying everything the software needs to deliver.
Don't “Reinvent the Wheel”	Don't waste time or effort recreating things that already exist; reuse proven solutions.

Principle	What It Means
Minimize Intellectual Distance	Reduce the gap between the real-world problem and the software solution, so the design maps naturally onto the problem.
Exhibit Uniformity and Integration	The design should be consistent throughout, and its subsystems should combine cleanly into one coherent system.
Accommodate Change	Design so the software can adjust to changing user needs rather than resisting them.
Degrade Gently	The software should keep working reasonably well even when an error occurs during execution, rather than failing catastrophically.
Be Assessed for Quality	The design must be evaluated for quality during the process, not just at the end.
Be Reviewed to Discover Errors	The design should be reviewed by others so problems are caught while they're still cheap to fix.
Design Is Not Coding, Coding Is Not Design	Design is the logic that solves the problem; coding is the language used to implement that logic. They are distinct activities.

Source: *GeeksforGeeks* – “Principles of Software Design.”

3. Separation of Concerns, Modularity and Abstraction

These three ideas reinforce one another: split a system into distinct parts, group each part so it has one clear job, and let each part hide how it works behind a small, stable interface.

- **Separation of Concerns (SoC):** Divide a problem into distinct parts — for example data loading, computation and plotting/output — so each can be understood and changed independently. Teams that skip this find that a change in one part of the system causes issues in unrelated areas.
- **Modularity and high cohesion:** Decompose a program into modules (functions, classes or files) that each handle one logically related task. A module has high cohesion when its parts genuinely belong together and serve a single purpose — which makes it easier to read, test and reuse. Breaking complex functions into smaller, self-contained components also makes them reusable across different parts of a project.
- **Low coupling:** Coupling measures how much modules depend on each other. Keep it low by having modules interact through simple, stable interfaces rather than reaching into each other's internals, so a change in one place doesn't ripple through the rest of the code. High cohesion and low coupling tend to go together.
- **Abstraction and information hiding (encapsulation):** Abstraction means exposing what a component does while hiding how it does it. Callers depend on the interface — a function signature or a class's public methods — not on the implementation, so the internals can change without breaking callers.
- **Choosing good module boundaries:** A useful guideline (David Parnas) is to draw module boundaries around the design decisions most likely to change — a file format, a storage backend, a numerical method — and hide each decision behind an interface, so an expected change stays contained within one module. A quick test: can you describe what a module does in one sentence without mentioning how it works? If not, it may be doing too much or leaking its internals.

4. Designing for Testability and Maintainability

Testable code and maintainable code grow from the same habits: build small, decoupled, predictable units. Code with weak cohesion, tight coupling or hidden state is hard to test — and the same traits make it hard to maintain.

- **Prefer pure functions and determinism:** A pure function returns the same output for the same input and has no side effects. Such functions are the easiest things to test, since a test is just an input and an expected output, with no setup or teardown required.
- **Separate computation from I/O and side effects:** Keep file access, network calls, randomness and printing in thin wrappers, and put the real logic in pure functions that take data in and return a result. This lets you test the logic without touching the filesystem or network.
- **Inject dependencies instead of hard-coding them:** A function or object should receive its collaborators from the outside rather than constructing them internally, so a test can pass in a simple stand-in (a test double) in place of a database, model, or clock.
- **Avoid hidden global mutable state:** Functions that read or write module-level globals behave differently depending on what ran before them, making results order-dependent and tests flaky. Pass state in as arguments instead.

A quick check: if testing a function requires creating files, network access, or a specific global state first, that's a hint to extract the core logic into a pure function and push the side effects to the edges.

Source: Harvard Kempner Institute Software Engineering Handbook – “Testability and Maintainability.”

5. Designing for Reusability and Extensibility

Reusability and extensibility are about designing something once and adapting it to new situations without rewriting what already works.

- **Parameterize instead of hard-coding:** A one-off script with a baked-in path, threshold or constant works only once. Passing those values in as arguments lets the same function serve many datasets and settings.
- **Package genuinely reusable code:** When a function or module proves useful across projects, move it into an installable package so others can import it rather than copy it.
- **Design for extension with the Open/Closed Principle:** Software entities should be “open for extension, but closed for modification” (Bertrand Meyer, popularized by Robert C. Martin). New behavior should be addable without editing tested, working code — for example, an `evaluate()` function that takes a metric as a parameter lets a new metric be added without touching `evaluate()` itself.
- **Add behavior by passing in a function or strategy:** Instead of growing an `if/elif` chain each time a new case is needed, accept the variable behavior as a parameter. Configuration files serve the same goal for choosing options without code changes.

Don't over-generalize

Adding extension points you don't yet need is speculative complexity. Build the flexible version when a second real use case appears, not before.

Source: Harvard Kempner Institute Software Engineering Handbook – “Reusability and Extensibility.”

6. An Introduction to Design Patterns

A design pattern is a named, reusable solution to a commonly recurring design problem, popularized by the “Gang of Four” (Gamma, Helm, Johnson, Vlissides) in *Design Patterns: Elements of Reusable Object-Oriented Software* (1994). You don't need to memorize a catalog; a handful of patterns are genuinely useful for keeping code flexible.

Patterns fall into three families: creational patterns concern how objects are made, structural patterns concern how objects and classes are composed, and behavioral patterns concern how objects collaborate and divide responsibility. A few of the most commonly used:

Pattern	Family	Purpose
Strategy	Behavioral	Pass an interchangeable function or object as a parameter so an algorithm can vary independently of the code that uses it, instead of growing an if/elif chain.
Factory	Creational	Centralize object construction in one place (often selected by a name or config value), replacing conditionals scattered across the codebase.
Adapter	Structural	Wrap a third-party library or service behind your own small interface, so the rest of the code depends on your interface rather than the external one.
Decorator	Structural	Attach new behavior to an object dynamically by wrapping it, without modifying the object's own class.
Singleton	Creational	Ensure a class has only one instance and provide a single global point of access to it.

Patterns are shared vocabulary — naming a recurring structure lets you describe a design in a word or two (“pass in a strategy,” “use a factory”) instead of re-explaining the mechanics. But patterns are tools, not goals: reach for one when it removes real, present pain, not to make code look sophisticated. Forcing a pattern adds indirection you must read and maintain, working against simplicity.

Sources: Harvard Kempner Institute Software Engineering Handbook – “Design Patterns (Introductory Level)”; Medium – “Software Design Principles Every Programmer Should Know” (design pattern examples: Decorator, Singleton, Factory, Strategy, Proxy).

7. Managing Dependencies Between Components

Beyond basic dependency injection (covered in Clean Coding Practices), dependency management is an architectural concern: how your own components depend on one another, and how you keep external dependencies — third-party libraries, files, networks, databases — from leaking into your core logic.

- **Mind the direction of dependencies:** High-level logic should not depend on low-level details; both should depend on an abstraction, and details should depend on that abstraction rather than the reverse. Concretely, your core logic should depend on an interface you define, not on a specific client or file format.
- **Isolate volatile dependencies behind a thin wrapper (the Adapter pattern):** Wrap an external library or service behind your own small interface, so the rest of the code talks to your wrapper instead of the

third party directly. When you swap a backend or weather an API change, the edit touches one file rather than every call site.

- **Isolation aids testing:** Once the core depends on your own interface, a test can pass in a simple fake in place of a real database or network call.
- **Keep third-party specifics at the edges:** Confine library-specific calls and types to the wrapper rather than threading them through the core — the same idea behind ports-and-adapters (hexagonal) architecture.

A useful signal

If changing one third-party library forces edits across many files, that dependency isn't isolated. Funnel it through a single wrapper so the rest of the code depends on your interface instead.

Source: Harvard Kempner Institute Software Engineering Handbook – “Dependency Management and Isolation.”

8. Documentation as Part of Design

Documentation is part of designing software, not a chore bolted on at the end. Describing an interface in words is itself a design step — it forces you to decide what a component promises before you commit to how it works.

- **A docstring is the contract for an interface:** A function or method docstring should summarize its behavior and document its arguments, return values, side effects, exceptions raised, and any restrictions on when it can be called. That's the set of promises callers depend on.
- **Write the interface and its docstring first:** Stating the contract before the body clarifies the design while it's still cheap to change. If you can't describe the inputs, outputs and failure modes cleanly, the boundary isn't settled yet and implementation is premature.
- **Hard-to-document code is a design smell:** If a clear explanation needs many caveats or special cases, the unit is usually doing too much or leaking its internals — treat that friction as a prompt to simplify or split.
- **Record significant design decisions:** An Architecture Decision Record (ADR) is a short document that captures one decision with its context, decision and consequences, so the reasoning behind architecturally significant choices survives as the project evolves.

Source: Harvard Kempner Institute Software Engineering Handbook – “Documentation as Part of Design.”

9. Iterative Design and Refactoring

Good design is rarely reached in one attempt — it emerges as understanding of the problem grows. Refactoring is the disciplined way to improve a design after the fact, by changing the internal structure of code while leaving its observable behavior intact.

- **Design iteratively:** Start with the simplest thing that works and let the structure evolve as the code grows, rather than trying to anticipate the final design up front.
- **Refactoring preserves external behavior:** As Martin Fowler defines it, refactoring is “a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.” Clarity improves; results do not.

- **Tests are the safety net:** Refactoring is only safe when you can confirm behavior is unchanged, so run tests after each step.
- **Work in small, separate steps:** Make one small change at a time, and keep pure refactoring commits separate from commits that change behavior, so a regression is easy to locate and review.

A frequent first refactoring is Extract Function/Method: pulling a block or a dense, unclear expression into a well-named function so the intent is clear and the logic can be reused, with behavior unchanged.

Source: Harvard Kempner Institute Software Engineering Handbook – “Iterative Design and Refactoring.”

10. Designing APIs and CLIs

How others use your code — including your future self — depends entirely on the interface you expose. There are two common kinds: an API (the functions and classes a library offers to be imported) and a CLI (a command-line interface for running a tool from the shell). Both are contracts.

- **Name things clearly and consistently:** Use predictable, descriptive names and a consistent style across an interface, so callers can guess the next name from the last one. Names are part of the contract — changing one breaks callers.
- **Keep the public surface small:** Expose only what users need and give it sensible defaults, so the common call is short while rare options are still reachable. A smaller surface is easier to document, test and keep stable.
- **Mark internals private, keep the public part stable:** Prefix non-public names with an underscore and declare a module's public API explicitly. This frees you to change internals without breaking callers.
- **Fail with clear errors:** Validate inputs and raise specific exceptions with messages that say what was wrong and how to fix it, rather than failing deep inside with an obscure error.
- **For a CLI, use a real argument parser:** It validates input and generates help text for free, and should delegate to a plain function so the logic stays importable and unit-testable independent of the command line.

Source: Harvard Kempner Institute Software Engineering Handbook — “Designing APIs and CLIs for Research Tools.”

11. Layered Architecture

A layered architecture organizes a larger system into layers that each have a distinct responsibility, with dependencies running one way. A useful four-layer split:

- **Entry/Interface layer:** A CLI, UI, notebook, or API call — takes input and reports results.
- **Orchestration/Workflow layer:** Ties a run together by sequencing the pipeline's steps (load → compute → save).
- **Core logic layer:** Holds the actual work — models, business rules, algorithms — with no file paths, network calls, argument parsing, or printing inside it.
- **Infrastructure/Data layer:** Handles file and network I/O, storage, and external services.

Dependencies should point one way, toward the stable core: each layer depends on the layer below it, and lower layers should be agnostic of who calls them. Keeping the core free of I/O and UI makes it testable in isolation, reusable across multiple entry points, and portable between environments. A quick test of the layering: can you import and run the core logic in a test without any files, network, or command-line arguments? If not, interface or I/O code has leaked into the core. Note: don't over-layer a small script — layers earn their keep on systems that grow more than one entry point or data source.

Source: Harvard Kempner Institute Software Engineering Handbook — “Layered Architecture in Scientific Computing Software” (generalizes to any layered system).

12. Functional vs. Object-Oriented Design

Functional and object-oriented styles are two common ways to organize code, and neither is universally better — choose by fit, not fashion. Most real-world code is a blend of both.

- **Functional style:** Build behavior from functions that transform data, ideally taking inputs and returning outputs with no internal state. Pure functions are easy to test and to compose, and functional style favors immutability — not mutating data in place.
- **Object-oriented style:** Bundle related data with the operations that act on it into objects, using encapsulation to hide internal details and sometimes inheritance to share behavior. This fits well when an entity has identity, a lifecycle, or many operations over the same shared state.
- **A practical default:** Reach first for functions plus simple data containers. Elaborate class hierarchies are often over-engineering — deep inheritance is hard to follow and rarely pays off. Add a class only when shared state or identity genuinely makes it the simpler description. If a class has only a constructor plus one method you call once, a plain function is usually clearer.

Source: Harvard Kempner Institute Software Engineering Handbook – “Functional vs. Object-Oriented Design.”

13. Beyond the Code: Prototyping, UX and Validation

Design principles aren't only about code structure — they also govern how a system is validated with real users before and while it's built. Skipping this is a common way projects unravel: teams add feature after feature on a foundation that was never properly designed or tested with users, and the system becomes more complex and bug-prone with every change.

- **Prototype before committing to development:** Low-fidelity prototypes (simple wireframes/mockups) let you test ideas early and get feedback before writing production code. High-fidelity, interactive prototypes let users click through something that mimics the final product, validating core functionality before the actual coding begins. The goal is to fail fast and learn faster — the earlier a mistake is caught, the cheaper it is to fix.
- **Design confirmation — don't assume, validate:** Conduct usability tests where real users complete tasks using your prototypes; watching them struggle reveals blind spots the team hadn't considered.
- **Consistency:** A unified style guide/design system — consistent buttons, forms and navigation — makes an interface predictable and easier to learn.
- **Clarity over cleverness:** Fancy animations or unique layouts that seem impressive to the team often confuse users. Clear labels, familiar icons and intuitive navigation beat novelty.
- **Accessibility and feedback:** Design should be usable for everyone — readable fonts, high-contrast color schemes, screen-reader compatibility — and the system should always tell the user what's happening (on click, on submit, on error).
- **Iterate, don't settle:** The first version of a design is rarely perfect; treat it as a living, evolving piece of the project that's continually refined based on feedback.
- **Collaborate across design and development from day one:** Handing finished mockups to developers to implement in isolation tends to produce a final product that doesn't match the original vision. Involving both sides from planning through implementation keeps the design and the build aligned.
- **Plan for scalability from the start:** Designing only for the present, without considering future growth, tends to surface performance and scalability problems once the user base grows — whether through modular architecture, cloud infrastructure, or load balancing.

Source: Medium — “Building Software that Lasts: A Journey Through Design Principles.”

14. Quick-Reference Design Principles Checklist

A condensed checklist of the principles covered in this file (see the separate Clean Coding Practices checklist for SOLID/DRY/KISS/YAGNI-level items).

- ☐ Design is treated as a distinct activity from coding, with its own review step.
- ☐ The problem is decomposed by separation of concerns — each part has one job.
- ☐ Modules show high cohesion and low coupling; boundaries sit around what's likely to change.
- ☐ Internals are hidden behind small, stable interfaces (abstraction/information hiding).
- ☐ Core logic is built from pure, testable functions; I/O and side effects are pushed to the edges.
- ☐ Dependencies are injected, not hard-coded, so tests can substitute fakes.
- ☐ Reusable logic is parameterized and packaged rather than duplicated per use case.

- ☐ New behavior can be added without editing tested code (Open/Closed).
- ☐ A design pattern is used only where it removes real, present pain — not for its own sake.
- ☐ External/third-party dependencies are isolated behind a wrapper or adapter.
- ☐ Interfaces (docstrings, APIs, CLIs) are written and reviewed as contracts, before or alongside implementation.
- ☐ Significant design decisions are recorded (e.g., in an ADR) with their reasoning.
- ☐ The design evolves through small, test-covered refactoring steps, not one big upfront attempt.
- ☐ Public APIs/CLIs have a small surface, consistent naming, sane defaults and clear error messages.
- ☐ Layers (interface, orchestration, core, infrastructure) depend one way, toward a stable core.
- ☐ The choice between functional and object-oriented style fits the problem, not habit.
- ☐ Ideas are prototyped and validated with real users before/while building, then iterated.
- ☐ The design accounts for future scale, not just today's requirements.

Compiled from: GeeksforGeeks – “An Introduction to Software Development Design Principles” and “Principles of Software Design”; Medium – “Building Software that Lasts” and “Software Design Principles Every Programmer Should Know”; and the Harvard Kempner Institute Software Engineering Handbook – “Software Design Principles” (Sections 6.1–6.13). Content already covered in the Clean Coding Practices file (SOLID, DRY, KISS, YAGNI, composition over inheritance, basic dependency injection, Law of Demeter, Boy Scout Rule) was deliberately left out to avoid duplication. No content beyond these sources has been added.